

RAG Architecture, Agent Comparisons & GitHub Models

How GitHub avoids sending whole repos to the model, the agentic landscape, and the model catalog

TOPICS COVERED

- › Copilot Enterprise Two-Stage RAG
- › Chunking & Hybrid Retrieval
- › Agent Planning & Tool Use
- › Claude Code / Codex / Gemini CLI Comparison
- › GitHub Models Architecture
- › Prompt Playground & Evaluation
- › Neighboring Tabs Context
- › Freshness & Incremental Indexing
- › Sub-Agents & Fleet Mode
- › Cursor & Windsurf Positioning
- › Model Catalog & Azure AI Compatibility
- › Enterprise Governance for Models

PART 4 — AI Retrieval Architecture

4.1 Why You Can't Send a Whole Repository to the Model

Even at the largest commercially available context windows (1M tokens in some providers' offerings), an enterprise monorepo with millions of lines of code, years of commit history, thousands of issues, and extensive documentation vastly exceeds what any model can process in a single call — and even where it technically fits, doing so is prohibitively slow and expensive, and dilutes the model's attention across mostly irrelevant material. The entire discipline of context engineering exists to solve a narrower problem: given a specific prompt, what is the minimum, highest-signal slice of the repository that should be assembled into the prompt.

4.2 GitHub's Documented Two-Stage RAG Architecture (Copilot Enterprise)

GitHub has published a direct description of how Retrieval-Augmented Generation works in Copilot Chat on GitHub.com. When a developer asks a question about a repository, RAG in Copilot Enterprise uses an internal search engine to find relevant code or text from indexed files to answer that question; that internal search engine performs a semantic search by analyzing the content of indexed documents and ranking them by relevance. Copilot Chat then performs a second semantic search — this time within the top-ranked documents — to find and retrieve the most relevant snippets, which are the actual text added to the prompt sent to the model.

✓ **VERIFIED** — Two-stage retrieval (corpus-level ranking, then snippet-level retrieval within top documents) per GitHub's own engineering blog, 'What is retrieval-augmented generation, and what does it do for generative AI?'

```
# Verified two-stage RAG pipeline (GitHub Copilot Enterprise,
# per GitHub's own engineering blog):

Developer question
  ▼
STAGE 1 – Corpus-level semantic search
  Internal search engine analyzes indexed repository content
  Ranks documents/files by relevance to the question
  ▼
STAGE 2 – Snippet-level semantic search (RAG proper)
  Within the TOP-RANKED documents only, perform a second
  semantic search to find the most relevant snippets
  ▼
Selected snippets inserted into the model prompt
  ▼
GitHub Copilot Chat generates a grounded response
```

4.3 Neighboring Tabs — A Verified, Simpler Retrieval Mechanism

Separately from the indexed-search pipeline, GitHub Copilot also processes code from a developer's other open editor tabs — a mechanism GitHub calls 'neighboring tabs' — to find and potentially add relevant information to the prompt. When a developer has many tabs open, Copilot scans the most recently reviewed ones rather than all of them, which is itself a context-budget decision: recency is used as a cheap, zero-latency relevance proxy that

requires no indexing or embedding computation at all.

✓ **VERIFIED** — 'Neighboring tabs' mechanism and recency-based tab selection, per GitHub's own engineering blog on RAG

■ *Neighboring tabs is architecturally distinct from, and complementary to, the indexed RAG pipeline: it requires no precomputed index and works even on a freshly opened, never-indexed file, making it a fast first line of context before falling back to heavier retrieval.*

4.4 Chunking and Embedding Quality as a First-Class Engineering Investment

As covered in Part 3, GitHub's 2025 embedding model upgrade delivered a documented 37.6% lift in retrieval quality with roughly 2x throughput and an 8x smaller index — concrete evidence that GitHub treats the chunking and embedding layer as core infrastructure worth dedicated optimization, not an implementation detail bolted onto a single model release.

Independent reverse-engineering of the open-source VS Code Copilot Chat client indicates local embedding chunks are sized at roughly 100-250 tokens, a range broadly consistent with general industry practice for code retrieval (small enough to be a coherent unit — a function, a class method — but large enough to carry meaningful semantic content).

■ **INFERRED** — Specific chunk size figures (100-250 tokens) come from independent source-code analysis, not GitHub's own published documentation; treat as a credible estimate of one client implementation, not a platform-wide constant.

4.5 Freshness and Incremental Indexing

A retrieval index is only useful if it reflects the current state of the repository. GitHub's local SQLite-backed embedding cache (per independent reverse-engineering) keys cache entries by file URI plus content version, meaning only changed files require re-chunking and re-embedding on each indexing pass — a standard incremental-indexing pattern also seen in comparable open tools, such as the elastic/semantic-code-search-indexer project, which explicitly markets incremental updates that only reprocess files changed since the last indexed commit.

■ **INFERRED** — The specific cache-invalidation key (URI + content version) is drawn from independent reverse-engineering of client source; the general pattern of incremental, change-driven re-indexing is corroborated by comparable open-source implementations and is standard practice across the industry.

4.6 Hybrid Search — Lexical + Semantic

The reverse-engineered, four-tier Copilot retrieval cascade (Part 3.2) effectively implements a hybrid-search posture: a fast local lexical/keyword strategy runs alongside or as a fallback to semantic embedding search, giving the system both precision on exact-term queries (symbol names, error strings) and recall on conceptual queries ("how is authentication handled"). This mirrors the architecture independently documented by comparable tools — for example, hybrid search combining semantic similarity vectors with BM25 keyword matching, fused via reciprocal rank fusion, followed by a deterministic rerank step, is an explicitly documented pattern in other code-search systems and academic RAG literature.

■ **INFERRED** — That GitHub's specific implementation performs rank fusion or reranking identically to these comparable systems is not confirmed by GitHub; the hybrid lexical+semantic posture itself is well-evidenced by the reverse-engineered cascade, but the precise fusion/reranking algorithm is inferred from industry-standard

technique, not GitHub documentation.

4.7 Multi-Repository, PR, Issue, and Commit Retrieval

The Coding Agent's documented scope (Part 2.3) is explicitly bounded to a single repository: it can explore linked issues and past pull requests via the built-in GitHub MCP server, but only within the current repository — multi-repository retrieval across an organization's full repository set is, by design, not part of the agent's default context-gathering behavior. This is a deliberate security and complexity boundary, not an oversight: unrestricted cross-repository retrieval would dramatically expand both the attack surface for data exfiltration and the cost/latency of context assembly.

✓ VERIFIED — Single-repository scope as a hard, documented constraint, per GitHub Docs 'Using Copilot coding agent'

PART 5 — AI Agent Architecture

5.1 Anatomy of an Agentic Loop

Across virtually every modern coding agent — GitHub Copilot Agent Mode, Anthropic's Claude Code, OpenAI's Codex CLI, Google's Gemini CLI — the core loop is structurally similar: the model is given a goal plus a set of callable tools (file read/write, terminal execution, test running, web/MCP access), it proposes the next action, the runtime executes it and returns the result, and the model evaluates the result and either continues or terminates. What differs substantially across vendors is the degree of autonomy, sandboxing strictness, and the surface area on which the loop runs (local IDE process vs. isolated cloud container vs. terminal process with direct host access).

5.2 GitHub's Specific Implementations

Mode	Where the loop runs	Approval model	Network access
Agent Mode (IDE)	Local IDE process	Terminal commands require approval; every tool call shows in IDE config support	Local host only, no internet
Coding Agent (cloud)	Ephemeral GitHub Actions runner	Triggered only by users with write access; PRs are reviewed by developers	Based on repo's default allowlist-based network access
Copilot CLI (GA Feb 2026)	Terminal process	Plan mode (strategy first) / Autopilot (no confirmation)	Hosted fleet mode (parallel sub-agents)

✓ VERIFIED — Copilot CLI's plan/autopilot/fleet mode trio and its February 2026 GA date, per multiple 2026 third-party technical guides synthesizing GitHub's own changelog and CLI documentation

5.3 Sub-Agents and Parallelism

GitHub Copilot's customization architecture documents a layered model — instructions establish identity, skills provide capability, agents define role, and hooks ensure safety — and supports custom sub-agents with isolated context, synchronous blocking behavior relative to the parent agent, and parallel execution, alongside the open Agent Skills standard (originally created by Anthropic) that now works across VS Code, Copilot CLI, Copilot Coding Agent, and Claude Code itself.

✓ VERIFIED — Layered customization architecture (Instructions/Skills/Agents/Hooks) and sub-agent isolation properties, per GitHub's published Copilot Customization Architecture documentation, with corroboration from VS Code's own blog on sub-agents and the unified multi-agent experience (Nov 2025–Feb 2026)

■ The convergence on Agent Skills as a cross-vendor open standard — usable by VS Code, Copilot CLI, Copilot Coding Agent, and Claude Code — is a notable, verified instance of the major agentic coding vendors standardizing a shared interface rather than each maintaining an incompatible proprietary format.

5.4 Comparison: GitHub Copilot vs. Codex vs. Claude Code vs. Gemini CLI vs. Cursor/Windsurf

This is one of the fastest-moving comparison spaces in software engineering, with vendors shipping major capability and pricing changes on a roughly monthly cadence through 2025-2026, and most head-to-head benchmark numbers in circulation are vendor-published or third-party-tested without independent verification. The comparison below should be read as a snapshot of architectural posture and publicly reported positioning, not a settled ranking.

■ **CONTESTED / RECENT** — Benchmark scores cited by third parties (Terminal-Bench, SWE-bench Verified/Pro) are frequently self-reported by vendors on leaderboards with no independent verification; one widely cited 2026 source explicitly notes that on the SWE-bench-style llm-stats leaderboard, all entries are vendor self-reported and zero are independently verified. Treat all specific percentage scores in this section as directional, not authoritative.

Tool	Vendor	Primary surface	Architectural emphasis (as reported)
GitHub Copilot (Agent Mode + Coding Assistant)	GitHub/Microsoft	IDE + cloud (GitHub-native)	Deepest native GitHub workflow integration (issues, PRs, Actions)
Claude Code	Anthropic	Terminal	Agentic search of entire codebase without manual context selection
Codex CLI	OpenAI	Terminal + cloud + IDE	Strongest sandboxing reported (Docker-based isolation by default)
Gemini CLI	Google	Terminal	Largest standard context window (1M tokens) reported as the product
Cursor	Anysphere	IDE (VS Code fork)	Polished in-editor multi-model experience; in-house 'Composer' tool
Windsurf	Cognition (post-acquisition)	IDE	Similar 2026 rebrand activity reported; similar 2026 rebrand activity positioning to Cursor; substantial 2026 product coverage

■ **CONTESTED / RECENT** — Windsurf's reported 2026 rebrand to 'Devin Desktop' following a Cognition acquisition, and similar fast-moving vendor/product changes (Gemini CLI's transition toward 'Antigravity CLI', GitHub's switch to usage-based billing) are drawn from very recent (within-weeks) third-party reporting at the time of writing and should be independently verified, as this market segment is changing faster than this report can be kept current.

5.5 The Consistent Differentiator: GitHub-Native Workflow Integration

Across third-party comparisons, the one architectural claim that recurs consistently and is also independently verifiable against GitHub's own documentation is that Copilot's coding agent has native, first-class integration with GitHub issues, pull requests, GitHub Actions, and (per GitHub's own announcement) external trackers including Azure Boards, Jira, Linear, Slack, and Teams — whereas competing terminal agents generally treat GitHub as one of several possible Git remotes rather than as the operational substrate the agent is built on top of (e.g., using GitHub Actions compute directly for its execution environment).

✓ **VERIFIED** — Coding agent's native GitHub Issues/Actions integration and external tracker integrations (Azure Boards, Jira, Linear, Slack, Teams), per GitHub's Build 2025 press release and subsequent 2026 product coverage

PART 6 — GitHub Models Platform

6.1 Architecture and Purpose

GitHub Models is explicitly positioned by GitHub as a suite of developer tools taking a developer 'from AI idea to ship,' including a model catalog, prompt management, and quantitative evaluations, embedding AI development directly into familiar GitHub workflows rather than requiring a separate Azure subscription or API key provisioning to begin experimenting.

✓ **VERIFIED** — Stated purpose and zero-friction onboarding design intent, per GitHub Docs 'About GitHub Models' and GitHub's original launch blog post

6.2 Model Catalog and Azure AI Compatibility

The catalog spans models from multiple publishers — at launch this included Llama 3.1, GPT-4o and GPT-4o mini, Phi 3, and Mistral Large 2, with the catalog continuing to expand (e.g., Grok 3 from xAI was added by mid-2025, and the catalog's REST API as of 2026 documentation lists entries such as GPT-4.1 with explicit capability metadata: streaming and tool-calling support, multi-modal input (text, image, audio), and a documented 1,048,576-token maximum input length for that model). Each catalog entry exposes a structured capability and limits schema via the REST API, making programmatic model selection and routing straightforward to implement.

✓ **VERIFIED** — Catalog publisher diversity and the structured JSON schema (id, publisher, registry, capabilities, limits, rate_limit_tier, supported modalities) per GitHub's own REST API documentation for the models catalog

```
# Verified: GitHub Models catalog REST endpoint and response shape
curl -L \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: Bearer <YOUR-TOKEN>" \
  -H "X-GitHub-API-Version: 2026-03-10" \
  https://models.github.ai/catalog/models

# Example entry (abbreviated, per GitHub Docs):
[
  {
    "id": "openai/gpt-4.1",
    "name": "OpenAI GPT-4.1",
    "publisher": "OpenAI",
    "registry": "azure-openai",
    "capabilities": ["streaming", "tool-calling"],
    "limits": {
      "max_input_tokens": 1048576,
      "max_output_tokens": 32768
    },
    "rate_limit_tier": "high",
    "supported_input_modalities": ["text", "image", "audio"],
    "supported_output_modalities": ["text"]
  }
]
```

The API surface is intentionally Azure AI Inference API-compatible: the architecture's central design principle is minimal transition cost between experimentation and production — the same SDK calls used in the free GitHub Models playground/Codespaces environment carry forward into a production Azure AI deployment with only endpoint and credential changes, not a logic rewrite.

✓ **VERIFIED** — Azure AI Inference API compatibility and the 'minimal transition cost' design principle, per GitHub's launch blog and independent technical write-up corroborating the same Azure AI Foundry glide path

6.3 Prompt Playground and Prompt-as-Code

The interactive playground lets users adjust model parameters (temperature, maximum tokens), switch between Chat and Code views, and compare two models side-by-side on the same prompt — explicitly designed as a low-friction, rate-limited environment for experimentation without local setup, prior to any production commitment.

GitHub Models supports storing prompts directly inside GitHub repositories, which GitHub frames as enabling automated text summarization and other AI-driven functionality to live in version control alongside the application code that consumes them — a direct instantiation of "prompt as code."

✓ **VERIFIED** — Repository-stored prompts and side-by-side model comparison, per GitHub Docs 'About GitHub Models'

6.4 Evaluation Framework

GitHub Models documentation explicitly lists 'quantitative evaluations' as a first-class capability alongside the catalog and prompt management, including the ability to test and compare AI model outputs using evaluators and scoring metrics, and to run prompt evaluations from GitHub Actions by piping a series of JSON test-case files into the GitHub Models command via the GitHub CLI — i.e., evaluation-as-CI, not a separate manual process.

✓ **VERIFIED** — Actions-integrated, JSON-test-case-driven evaluation workflow via gh CLI, per GitHub's original GitHub Models launch blog post

6.5 Enterprise Governance for GitHub Models

GitHub explicitly documents organization- and repository-level controls to secure and govern AI models in use, distinct from general Copilot policy controls — implying a deliberate design choice to let enterprises restrict which models, at the catalog level, developers in their org may invoke, separate from which Copilot product features are enabled.

■ **INFERRED** — The precise mechanics of model-level allowlisting (e.g., whether it is enforced per-organization, per-repository, or both, and how it composes with Copilot's own model-selection policy) are referenced by GitHub Docs section titles but not detailed in the search results reviewed for this report; readers implementing enterprise governance should consult GitHub's live documentation directly rather than relying on this summary.

6.6 Privacy Posture for GitHub Models

GitHub states explicitly that no prompts or outputs submitted through GitHub Models are shared with the underlying model providers, nor used to train or improve those models — a privacy commitment stated at the launch of the product and structurally consistent with GitHub's broader Business/Enterprise Copilot data-handling posture (see Part 11 for the more complex and recently contested situation around individual-tier Copilot data use).

✓ **VERIFIED** — No-sharing, no-training commitment for GitHub Models specifically, per GitHub's original launch announcement

Key Takeaways — Parts 4–6

- GitHub's RAG pipeline for Copilot Enterprise is a genuinely two-stage process — corpus-level document ranking, then snippet-level retrieval within the top-ranked documents — and this is one of the better-documented architectural claims in the entire report, sourced directly from GitHub's own engineering blog.
- 'Neighboring tabs' is a separate, simpler, zero-indexing context source that runs alongside the heavier RAG pipeline, illustrating that GitHub's actual production system is a cascade of multiple techniques rather than a single retrieval algorithm.
- The competitive agentic-coding landscape (Copilot, Claude Code, Codex CLI, Gemini CLI, Cursor, Windsurf) is changing on a near-monthly cadence; any specific benchmark percentage or pricing figure should be treated as a snapshot, not a stable fact, and self-reported vendor benchmarks should be read with explicit skepticism.
- GitHub Models is architected for minimal-friction graduation from free experimentation to production Azure AI deployment — the Azure AI Inference API compatibility is the single most consequential design decision in the platform, and is well-verified.